

# Compte-rendu de TP4 : Horloge Réseau (ESP32)

**Nom :** [Votre Nom]

**Classe :** CIEL IR

**Sujet :** Récupération de l'heure via NTP et affichage sur écran OLED

## 1. Introduction

L'objectif de ce TP est de mettre en œuvre les fonctionnalités réseau de l'ESP32 pour concevoir une horloge synchronisée sur internet. Nous avons appris à manipuler des objets C++, à gérer la connectivité Wi-Fi et à utiliser le protocole NTP.

## 2. Activité 1 : Connexion de la carte au réseau

### 1.1 Récupération de l'adresse MAC

**Question (01) : Modes de programmation Wi-Fi**

- **Mode STA (Station) :** L'ESP32 agit comme un client qui se connecte à un point d'accès existant.
- **Mode AP (Access Point) :** L'ESP32 agit lui-même comme un point d'accès.

**Question (02) : Choix du mode** Nous utilisons le **mode STA**.

**Question (03) : Obtention de l'adresse MAC**

L'adresse MAC de ma carte est : **24:6F:28:22:72:C8**.

## 3. Activité 2 : Récupération de la date et l'heure (NTP)

### 2.2 Affichage sur écran OLED (Cahier des charges final)

**Question (09) : Code final non-bloquant** Pour respecter les bonnes pratiques du développement embarqué, nous avons remplacé les instructions bloquantes (`delay`, `while`) par une machine à états pilotée par `millis()`. Cela permet à l'affichage de rester réactif même pendant les phases de connexion.

```
#include <Arduino.h>
#include <WiFi.h>
#include <Wire.h>
```

```

#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include "NTP_Client.hpp"

#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, -1);

// Configuration NTP (France : UTC+1, Heure d'été activée)
NTP_Client ntp(1, true, "pool.ntp.org");

// États du système
enum State { IDLE, START_WIFI, WAIT_WIFI, WAIT_NTP, DISCONNECT };
State currentState = START_WIFI;

unsigned long lastSyncMillis = 0;
unsigned long lastDisplayMillis = 0;
unsigned long stateTimeout = 0;
const unsigned long SYNC_INTERVAL = 30 * 60 * 1000; // 30 minutes

void handleSyncStateMachine() {
    switch (currentState) {
        case IDLE:
            // Attente de l'intervalle de 30 minutes
            if (millis() - lastSyncMillis >= SYNC_INTERVAL) {
                currentState = START_WIFI;
            }
            break;

        case START_WIFI:
            Serial.println("Démarrage Wi-Fi...");
            WiFi.mode(WIFI_STA);
            WiFi.begin("SNIR", "");
            stateTimeout = millis();
            currentState = WAIT_WIFI;
            break;

        case WAIT_WIFI:
            if (WiFi.status() == WL_CONNECTED) {
                Serial.println("Wi-Fi connecté. Maj NTP...");
                ntp.update();
                stateTimeout = millis();
            }
    }
}

```

```

        currentState = WAIT_NTP;
    } else if (millis() - stateTimeout > 10000) { // Timeout 10s
        Serial.println("Échec Wi-Fi.");
        currentState = DISCONNECT;
    }
    break;

case WAIT_NTP:
    struct tm timeinfo;
    if (getLocalTime(&timeinfo)) {
        Serial.println("Heure reçue !");
        lastSyncMillis = millis();
        currentState = DISCONNECT;
    } else if (millis() - stateTimeout > 5000) { // Timeout 5s
        Serial.println("Échec NTP.");
        currentState = DISCONNECT;
    }
    break;

case DISCONNECT:
    WiFi.disconnect(true);
    WiFi.mode(WIFI_OFF);
    Serial.println("Wi-Fi désactivé.");
    currentState = IDLE;
    break;
}
}

void updateDisplay() {
    if (millis() - lastDisplayMillis >= 1000) { // Rafraîchissement 1Hz
        lastDisplayMillis = millis();
        display.clearDisplay();

        // Zone Jaune : Date
        display.setTextSize(1);
        display.setTextColor(SSD1306_WHITE);
        display.setCursor(10, 5);
        display.print(ntp.getDateLong(false));

        // Zone Bleue : Heure
        display.setTextSize(2);
        display.setCursor(15, 35);
    }
}

```

```

        display.print(ntp.getTime());

        // Indicateur de synchro en cours
        if (currentState != IDLE) {
            display.fillCircle(120, 5, 3, SSD1306_WHITE);
        }

        display.display();
    }
}

void setup() {
    Serial.begin(9600);
    if(!display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) for(;;);
    display.clearDisplay();
    display.display();
}

void loop() {
    handleSyncStateMachine(); // Gestion réseau asynchrone
    updateDisplay();           // Gestion affichage asynchrone
}

```

## 4. Conclusion

L'implémentation d'une machine à états a permis de supprimer toutes les instructions bloquantes. Le processeur peut désormais mettre à jour l'affichage chaque seconde de manière fluide, même pendant que la pile TCP/IP négocie la connexion Wi-Fi ou attend la réponse du serveur NTP.